LLM Prompting

Tuesday, March 24, 2026 12:12 PM

Temperature:
Temperature controls **how random or predictable the model's output is** by adjusting the probability distribution of next tokens. Lower temperature (e.g., **0–0.3**) makes the model pick high-probability words → **more accurate, consistent, factual**; higher temperature (e.g., **0.7–1.0**) increases randomness → **more creative, diverse, but riskier**.

Top-K and top-P

Top-K and Top-P are ways to control **which next words the model is allowed to choose from**.
With **Top-K**, you tell the model: "only pick from the best K token options," so **K=1** means always pick the single best token, while larger K allows more variety.
With **Top-P**, you tell the model: "keep adding the most likely tokens until their total probability reaches P," so it adapts dynamically—sometimes a small set is enough, sometimes it needs more.
Example: if the next-word probabilities are `cat 0.40, dog 0.30, bird 0.15, car 0.10, banana 0.05`, then **Top-K=2** allows only `cat, dog`, while **Top-P=0.85** allows `cat, dog, bird` because `0.40+0.30+0.15 = 0.85`.

Prompt engineering
-is the way we **write and structure instructions** so the LLM understands exactly what we want, how we want it, and in what format. We need it because a vague prompt gives vague, wrong, or incomplete answers, while a clear prompt improves **accuracy, relevance, tone, and output format**.

Zero-shot prompting
- means you ask the model to do a task **without giving any examples** first. You only give the instruction, and the model uses its pretrained knowledge to answer. We use it when the task is simple or already clear enough from the prompt. Example: **"Classify this review as positive or negative: 'The movie was boring and too long.'"** — no example is given, but the model can still answer **negative**.

One-shot prompting

means you give the model **one example** before asking it to do the real task.
Few-shot prompting

means you give **a few examples** so the model learns the pattern, format, or style before answering. We need them when zero-shot is not enough, especially for **format-sensitive or pattern-based tasks**.

Example:

- **One-shot:** "English: hello → French: bonjour. English: thank you → ?"
- **Few-shot:** "cat → animal, rose → flower, carrot → vegetable, eagle → ?" so the model learns the pattern and answers **bird/animal category** depending on task style.

System prompting
-means giving the model a **high-level instruction that sets its overall behavior, role, rules, and boundaries** before the user prompt is handled. We need it so the model stays consistent in **tone, task, safety, and style** across the whole conversation, instead of changing behavior with every message. It is like telling the model, **"You are a polite DBMS tutor. Explain in simple words, keep answers short, and give one example."** Then if the user asks, "What is normalization?", the reply will follow that style automatically.

Role prompting
- means telling the model to **act as a specific role or expert** so the answer comes in that perspective and style. We need it because the same question can be answered very differently by a **teacher, doctor, recruiter, or software engineer**. Example: if you ask **"Act as a Python teacher and explain loops to a beginner with one simple example,"** the answer becomes more beginner-friendly than just saying **"Explain loops."**

Contextual prompting

- means giving the model **background information, situation, or constraints** along with the question so it can answer more accurately. We need it because without context, the model may give a generic answer that does not fit your exact need. Example: instead of saying **"Write an email,"** say **"Write a formal email to my professor asking for a 2-day extension because I was sick"**—the extra context helps the model give the right tone and content.

Step-back prompting

- means asking the model to first think about the **broader principle or high-level idea** before solving the exact question. We need it because jumping directly to the answer can make the model miss logic, context, or better reasoning. Example: instead of asking **"How do I design a fraud detection model for my app?"**, first ask **"What are the general principles of fraud detection systems?"** and then ask for the specific design—this often gives a stronger final answer.

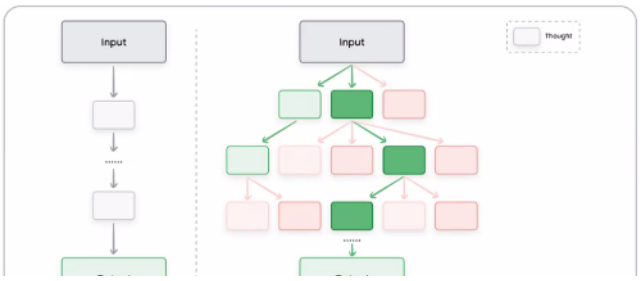
Chain of Thought (CoT)

- is a prompting method where the model is encouraged to **reason step by step** before giving the final answer. We need it for tasks that require **logic, math, multi-step analysis, or decision-making**, because it reduces mistakes from jumping too quickly to an answer. Example: if asked **"A shop had 120 items, sold 35, then added 20. How many now?"**, CoT would follow: `120 - 35 = 85`, then `85 + 20 = 105`, so the answer is **105**. In simple words, it is like telling the model: **"Don't just answer — think in steps."**

Prompt	`When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner? Let's think step by step.`
Output	1. Let's assume my current age is "x" years. 2. When I was 3 years old, my age was 3 years. 3. My partner's age at that time was 3 times my age, which means my partner's age was 3 * 3 = 9 years. 4. Now, I am 20 years old, which means my age has increased by 20 - 3 = 17 years since I was 3 years old. 5. Since my partner's age was 9 years when I was 3 years old, and my age has increased by 17 years, my partner's age must have also increased by 17 years. 6. Therefore, my partner's current age is 9 + 17 = 26 years. So, my partner is 26 years old.

Tree of Thoughts (ToT)

Zis a prompting method where the model explores **multiple possible reasoning branches like a tree**, checks which branch looks best, and then continues from the strongest one. We need it for **complex problems** where one straight step-by-step path may get stuck or go wrong, because ToT lets the model **compare options before committing**. Example: in a puzzle, instead of trying only one solution path, the model tries **Path A, B, and C**, sees that **B** is making more sense, and continues from **B** to reach the final answer. It is like solving a maze by checking a few routes first instead of blindly following one path.



Self-consistency

- is an advanced prompting method where the model generates **multiple different reasoning paths** for the same question and then chooses the **most common or most consistent final answer**. We need it because one single chain of thought can be wrong, but if several reasoning paths reach the same answer, confidence is higher. Example: for a math word problem, the model may solve it 5 different ways; if 4 of them give **42**, then **42** is chosen as the final answer. It is like asking **many students to solve the same problem and trusting the answer most of them agree on**.

ReAct (Reason + Act)

-is a prompting method where the model does **two things together**: it first **reasons about what to do**, and then it **takes an action** such as searching, using a tool, retrieving data, or asking for the next step. We need it because some problems cannot be solved from memory alone—they need both **thinking and doing**, so the model can gather information before answering. Example: if the task is **"What's the weather in New York and should I carry an umbrella?"**, the model first reasons **"I need current weather data"**, then acts by checking the weather, then reasons again and answers **"Yes, carry an umbrella because rain is expected."** It is like a student who does not just think in their head, but also opens the book, checks the data, and then gives the answer.

Automatic Prompt Engineering (APE)

-means using a model or program to **automatically create, test, and improve prompts** instead of writing all prompts manually. We need it because the best prompt is often not obvious, and APE helps find prompts that give **better accuracy, consistency, and task performance** with less human trial and error. Example: if the task is sentiment classification, instead of a person manually trying prompts like **"Classify this review"** or **"Tell whether this is positive or negative,"** the system generates many prompt versions, tests them on sample data, and keeps the one that performs best.

Code prompting

means writing prompts specifically to make the model **generate, explain, debug, or optimize code**. We need it because normal prompts give generic answers, but structured code prompts produce **correct, clean, and usable code outputs**.

How we communicate in code prompting:

- Specify **language + task + constraints + format**
- Optionally include **input/output examples**

Example:

"Write a Python function to check if a number is prime. Use time complexity O(√n), include comments, and show one example input/output."

→ This gives much better code than just saying "write prime code."

Guardrail prompting

-means adding **clear rules, limits, and boundaries** in the prompt so the model avoids unwanted, unsafe, off-topic, or wrong-style answers. We need it to keep the output **safe, relevant, and controlled**, especially in sensitive tasks like medical, legal, coding, or customer support. Example: **"Answer as a study tutor, explain in simple words, do not give harmful advice, do not make up facts, and say 'I don't know' if uncertain"**—these are guardrails that shape the response. It is like putting **road barriers on a highway** so the model stays in the correct lane.

Prompt injection

- is when a user or external content tries to **trick the LLM into ignoring its original instructions** and doing something unintended, which OWASP describes as a major LLM security risk.

Grounding

- means making the LLM answer based on **real, relevant external information** instead of only relying on what it remembers from training. We need it because the model's memory can be outdated, vague, or wrong, but grounding ties the answer to a **source like a document, database, website, or retrieved context**. Example: if you ask **"What does my lease say about pets?"**, a grounded system first reads **your actual lease file** and then answers from that, instead of giving a general guess about pet policies. In simple words, grounding is **connecting the model to facts before it answers**.